

Definição formal dos autômatos - (V, S, q_0, F, d)

V = vocabulário S = conjunto de estados q_0 = estado inicial F = conjunto de estados finais

d = função de transição $d(q, a) = p$

Determinístico $\rightarrow$ cada estado tem só 1 transição p/ cada símbolo. Não-determinístico $\rightarrow$ quer a resposta

ALG1 - Conversão de não-determinístico em determinístico:

1. Começa com o estado inicial ou concatenação de estados que se pode chegar num outro algum símbolo (transições ϵ). Verifica então, para cada símbolo, quais estados não alcança, e cria um estado concatenando todos eles. Ex: $d(q_0, a) = \{q_1, q_2\} \rightarrow$ cria estado $q_1 q_2$
2. Assim cada estado mantém suas transições determinísticas e faz as não-determinísticas e faz a concatenação em cada
3. Para os estados compostos, as transições são as concatenações dos estados individuais que os formam

ALG2 - Minimizar estados de um autômato finito determinístico:

1. Separa os estados em dois grupos inicialmente: F (estados finais) e NF (estados não-finais)
2. Para cada grupo, se analisa como cada estado se comporta p/ cada símbolo. Se houverem diferenças (erro, terminam em grupos diferentes, etc) precisam ser separados.
3. Repete até não poder mais separar os estados. O grupo com q_0 é o mesmo inicial e os outros com finais

gramáticas $\rightarrow$ variáveis terminais (símbolos do vocabulário) e não-terminais (geram produção)
tipos: sem restrições. Ex: $aAbB \rightarrow abab$; restrições de contexto: produção de que produzidos;
Ex: $AB \rightarrow ab$ e a

limites de contexto: só um símbolo na origem da produção. Ex: $A \rightarrow BXA$

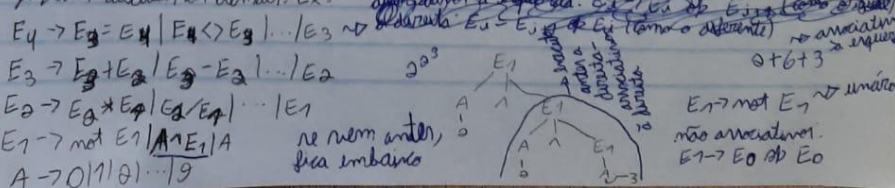
regulares: só um não-terminal na produção e um terminal produzido, podendo ter um não-terminal
Ex: $A \rightarrow aA$ e aB

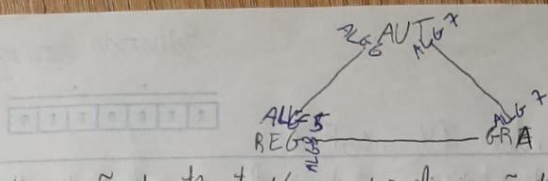
ALG3 - Remoção de recursão à esquerda:

1. Remove recursões indiretas substituindo símbolos não-terminais por suas produções em não-terminais
2. Remove recursões à esquerda ao separar a expressão em alfas (com recursão à esquerda) e betas (demais). Cria então duas produções: uma sendo a união dos betas seguintes da nova variável não-terminal e outra a união dos alfas com ϵ sendo os alfas seguintes de recursão da nova produção. Ex: $A \rightarrow sbAcd$ $\rightarrow$ $A \rightarrow \epsilon A'IE$

ALG4 - Remoção de ambiguidade (uma sentença pode ser derivada de mais de uma maneira pela gramática):

1. Cria uma produção p/ cada nível de prioridade, colocando as de maior prioridade antes (topo da árvore) e menor prioridade depois. As de menor prioridade serão então calculadas antes p/ poder calcular as demais. Ex:



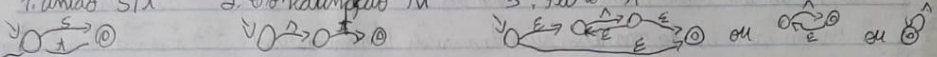


ALG 5 - Conversão de autômato f/ regex por eliminação de estados:

1. Unida os estados inicial e final, (1) se houver estados chegando ou (f) saindo.
2. Um por um, elimina os estados, criando novas transições com todas as combinações de entrada e saída que o teriam como intermediário. Fechos não incluem nas novas transições, no meio da concatenação da entrada e saída.

ALG 6 - Conversão de regex p/ autômato por Thompson

1. União s/t
2. Concatenação nt
3. Fecho nt*



ALG 7 - Conversão de autômato p/ gramática:

1. Cada estado do autômato é uma produção e símbolo não-terminal da gramática.
2. Transições tem o símbolo passado como símbolo terminal e o estado que recebe é um não-terminal concatenado. Se for uma transição f/ um estado final, não-terminal pode ser desprezado. Gramática gerada é regular.

ALG 8 - Conversão de gramática regular para regex:

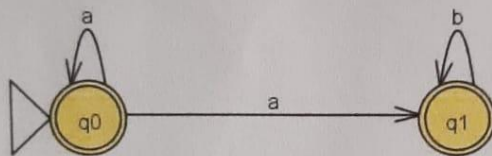
1. Cada produção vira um regex, e não terminais não sendo substituídos até o estado inicial ser um regex único.
2. Quando não há não-terminais (fora o próprio da produção) pode gerar o regex ao concatenar o fecho da união dos símbolos recursivos (remonendo a recursão) com a união dos não recursivos.

$E_3 \rightarrow E_0 \langle E_2 | E_0 \rangle E_2 | E_0$
 $E_2 \rightarrow E_0 + E_1 | E_2 - E_1 | E_1$
 $E_1 \rightarrow E_1 * E_0 | E_1 / E_0 | E_0$
 $E_0 \rightarrow C^{\wedge} E_0 | C$
 $C \rightarrow \text{const} | id | (E_3) | id(E)$
 $E \rightarrow E_3 L | \epsilon$
 $L \rightarrow E | \epsilon$

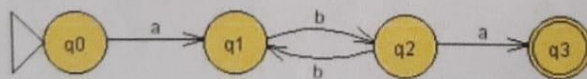
1)(1 ponto) Escreva uma expressão regular sobre o alfabeto $\{a, b, c\}$ que gere a linguagem cujas sentenças contenham exatamente duas ocorrências do símbolo a .

2)(1 ponto) Encontre um autômato finito determinístico que reconheça a linguagem gerada pela expressão $a^*(a|\epsilon)b^*$.

3)(1.5 pontos) Converta o autômato a seguir para um autômato finito determinístico equivalente.



4)(1.5 pontos) Encontre a expressão regular equivalente ao autômato a seguir:



5)(1 ponto) Converta a gramática regular a seguir para uma expressão regular. Considere que o símbolo inicial da gramática é o símbolo S :

$$S \rightarrow 1S|2S|0C$$

$$B \rightarrow 0B|1C|2S$$

$$C \rightarrow 0B | \epsilon$$

6)(1 ponto) Elimine a recursão à esquerda (direta e indireta) da gramática a seguir:

$$S \rightarrow A0 \mid 0B \mid 11$$

$$A \rightarrow 01 \mid S0$$

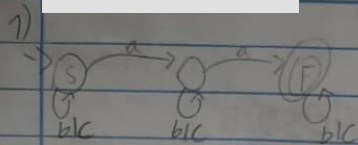
$$B \rightarrow B0 \mid 0A$$

7)(1.5 pontos) Construa uma gramática livre de contexto que gere a linguagem $L = \{w w^R\}$ onde w representa qualquer sentença sobre $\{a, b\}^*$ e w^R representa a sentença inversa a w , ou seja, os mesmos símbolos de w na ordem contrária. Assim, se $w=abb$, $w^R=bba$.

$\{a,b\}^* b \{a,b\}^*$

8)(1.5 pontos) Apresente uma gramática regular que gere a linguagem formada por todas as sentenças sobre $\{a, b, c\}^*$ onde o símbolo **a** não ocorre imediatamente à esquerda de um símbolo **b**.

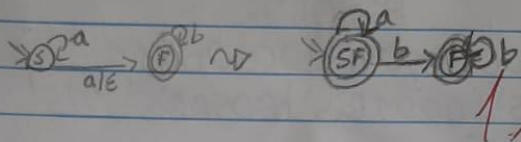
DEZ



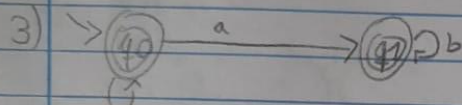
R: $(b|c)^* a (b|c)^* a (b|c)^*$ 1.5

$a b b c b b b c a b c b$

2) $a^* (a| \epsilon) b^*$

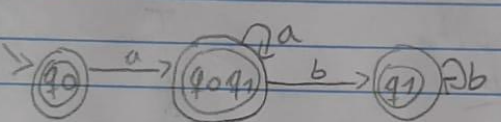


$(a, b) = \{FF\}$
 $(a, a) = \{SF\}$
 $(\epsilon, b) = \{FF\}$



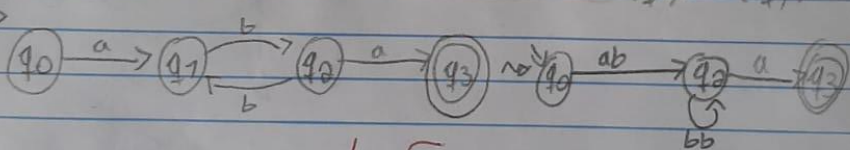
$a^* b^*$

1.5



$(q_0, a) = \{q_0, q_1\}$
 $(q_0, b) = \{q_1\}$
 $(q_1, a) = \{q_0, q_1\}$
 $(q_1, b) = \{q_1\}$

4)



$a b (b|a)^* a$ 1.5

5) $S \rightarrow 1S | 2S | 0C \leadsto S \rightarrow 1S | 2S | 00B | 0$

$B \rightarrow 0B | 1C | 2S \leadsto B \rightarrow 0B | 10B | 112S \leadsto B \rightarrow (0110)^* (112S)$

$C \rightarrow 0B | \epsilon$

$S \rightarrow 1S | 2S | 00(0110)^* (112S) | 0 \leadsto 1S | 2S | 00(0110)^* 1 | 00(0110)^* 2S | 0$
 $S \rightarrow (112 | 00(0110)^* 2)^* (00(0110)^* 1 | 0)$

1.5

000100

0100000

A00 500 A000

50000
0100000

00110000

000000 → 000000
000100

6) $S \rightarrow A0|OB|11 \rightsquigarrow S \rightarrow 010|500|OB|11$

$A \rightarrow 01|50$

$B \rightarrow B0|OA \rightsquigarrow B \rightarrow OAB' \rightsquigarrow B \rightarrow (001|050)B'$

$B' \rightarrow OB'|E$

$S \rightarrow 010|500|0001B'|0050B'|11$

R: $S \rightarrow 010S'|0001B'S'|0050B'S'|11S'$

$S' \rightarrow 00S'|E$

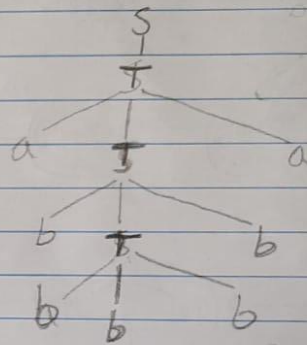
$B' \rightarrow OB'|E$

1.5

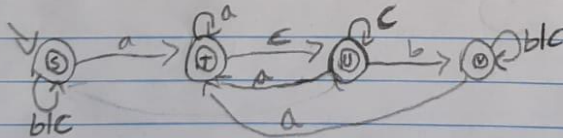
7) $S \rightarrow T|b$

$T \rightarrow aTa|bTb|aba|bbb$

1.5



8)



$S \rightarrow aT|bS|cS|E|c|E$

$T \rightarrow cU|aT|E$

$U \rightarrow bV|cU|aT|E$

$V \rightarrow bV|cV|aT|E$

1.5